Milestone 1 — System Connections and Interface Plan

Due: Saturday, Oct 3

Purpose of This Milestone

For Milestone 1, you are **not building the autonomous vehicle yet**.

You are deciding how the four parts of our vehicle will communicate with each other.

Our four subsystems are:

1. **Perception & Sensor Fusion**
2. **Motion Planning**
3. **Autonomous Vehicle Control**
4. **Hardware Integration & Active Safety**

Think of each subsystem as a box.

Each box:

- receives information,
- does something with that information, and
- sends information to another box.

For this milestone, your job is to identify:

What information does my subsystem need?

and

What information does my subsystem provide to the other teams?

That is what we mean by an **interface** or **interface contract**.

Important: What You Do NOT Need to Do Yet

For Milestone 1, you do **not** need to:

- build the complete vehicle,
- purchase new hardware,
- purchase new sensors,

- write Arduino code,
- create a complete Simulink model,
- create a complete Stateflow model,
- tune a PID controller,
- implement GPS,
- implement SLAM,
- implement computer vision,
- implement a complete waypoint-following algorithm,
- implement a complete path-planning algorithm,
- or have a working autonomous vehicle.

Those tasks come later.

For now, focus only on **how the pieces of the system will eventually connect**.

Overall System

We will begin with this basic architecture:

Sensors → Perception → Motion Planning → Vehicle Control → Motors/Steering

The **Hardware Integration & Active Safety** team will help connect the software to the physical vehicle and will also monitor the system for unsafe conditions.

The Safety system must be able to override normal vehicle operation when necessary.

For example:

Obstacle detected → Perception reports obstacle distance → Safety determines that the obstacle is too close → Emergency Stop becomes active → Vehicle stops

You do not need to invent a completely different architecture.

A Simple Example

Suppose a sensor detects an obstacle **0.75 meters** in front of the vehicle.

The Perception team may send:

obstacle_distance = 0.75

to the Safety team.

The Safety team may then decide:

emergency_stop = TRUE

The Control/Hardware system responds by stopping the vehicle.

For Milestone 1, you do **not** have to make this happen physically.

You only need to agree on things such as:

- What is the signal called?
- Which team sends it?
- Which team receives it?
- What does the value mean?
- What units are being used?

Starting Signals

You do not have to invent every signal from scratch.

Use the following table as a **starting point**.

You may add, remove, or modify signals if your teams determine that something different is needed.

Signal	**Sent By**	**Sent To**	**Meaning**
obstacle_distance	Perception	Planning / Safety	Distance to the closest detected obstacle
obstacle_detected	Perception	Planning / Safety	Indicates whether an obstacle is detected
sensor_valid	Perception	Safety	Indicates whether sensor information is valid
desired_speed	Motion Planning	Control	Speed the vehicle should try to maintain

desired_steering	Motion Planning	Control	Direction/steering command the vehicle should follow
motor_command	Control	Hardware	Command sent to the drive motor
steering_command	Control	Hardware	Command sent to the steering mechanism
emergency_stop	Safety	Control / Hardware	Stops or overrides normal vehicle operation
system_mode	Control / Safety	Other subsystems	Current state such as Normal, Stop, or Fault

These are **examples**. You are not required to use these exact signal names.

What Information Should Be Defined for Each Signal?

For each important signal that your subsystem sends or receives, provide the following information.

Information	**What It Means**	**Example**
Signal name	Name used for the information	obstacle_distance
Source	Team that produces the information	Perception
Destination	Team that receives the information	Safety
Description	What the signal represents	Distance to closest obstacle
Data type	Type of value	double
Units	Measurement units	meters
Expected range	Normal values you expect	0–4 m
Update rate	How often the value may be updated	Approximately 10 Hz
Invalid/fault condition	What happens if information is bad	NaN or sensor_valid = FALSE
Response if unavailable	What the system should do	Enter safe-stop mode

Do not worry if you do not yet know the exact update frequency or range.

Use a reasonable estimate and clearly identify anything that still needs to be confirmed.

Team 1 — Perception & Sensor Fusion

Your role

Your subsystem represents the **eyes and sensing part of the vehicle**.

It receives measurements from sensors and turns them into useful information for the rest of the vehicle.

For Milestone 1, answer these questions:

- What information might the vehicle need from its sensors?
- What information will your subsystem send to Motion Planning?
- What information will your subsystem send to Safety?
- How will another subsystem know if sensor information is valid?
- What happens if a sensor fails or gives an unreasonable value?

Possible information may include:

- obstacle distance,
- obstacle detected/not detected,
- sensor validity,
- vehicle speed,
- lane/path information if needed later.

Example

Input:

Ultrasonic sensor measurement

Output:

obstacle_distance = 0.75 m

You do **not** need to have the sensor physically working for Milestone 1.

You also do **not** need to select or purchase new sensors right now.

Use the hardware we already have as the starting point.

Team 2 — Motion Planning

Your role

Your subsystem decides **what the vehicle should do next**.

For example:

- continue straight,
- slow down,
- stop,
- turn,
- or move toward a target.

For Milestone 1, answer:

- What information do you need from Perception?
- What information will you send to Control?
- How will you represent desired speed?
- How will you represent desired steering?
- How might an obstacle affect your planned movement?

Example

Input:

obstacle_distance = 2.0 m

Output:

desired_speed = 0.5 m/s

or

desired_steering = 10 degrees

You do **not** need to build a complete path-planning algorithm yet.

What Is a Waypoint?

A **waypoint** is simply a target position.

For example, in simulation, a vehicle could be told to move through:

(0,0) → (1,0) → (1,1)

Those three points are waypoints.

For this project, waypoint behavior may initially be demonstrated in MATLAB/Simulink.

You do **not** need GPS or a sophisticated localization system for Milestone 1.

If our physical hardware does not support exact position measurement, the physical vehicle demonstration can use a simpler planned route.

What Is a Trajectory?

A trajectory describes how the vehicle should move toward a target.

For our project, do not overcomplicate this term.

The Motion Planning team may eventually produce simple information such as:

desired_speed = 0.5 m/s

and

desired_steering = 15 degrees

The Control team can then use those values to operate the vehicle.

For Milestone 1, you only need to decide **what information should be sent**.

You are not expected to calculate a complete trajectory yet.

Team 3 — Autonomous Vehicle Control

Your role

Your subsystem takes the requested vehicle behavior and turns it into commands for the motor and steering system.

For Milestone 1, answer:

- What information do you need from Motion Planning?

- Do you need information about the current vehicle state?
- What information will you send to the motor?
- What information will you send to the steering mechanism?
- What happens when the Safety team sends an emergency-stop command?
- What operating modes might the controller have?

Possible modes may include:

- Normal
- Slow
- Stop
- Emergency
- Fault

Example

Input:

desired_speed = 0.5 m/s

Output:

motor_command = 120

You do **not** need to design or tune the final PI/PID controller for Milestone 1.

That will happen later during modeling and simulation.

Team 4 — Hardware Integration & Active Safety

Your role

Your subsystem connects the software to the physical vehicle and makes sure the system can move into a safe state when something goes wrong.

For Milestone 1, answer:

- What vehicle hardware do we already have?
- What microcontroller/Arduino will be used?

- How might the motor connect to the controller?
- How might steering connect to the controller?
- What information does the hardware need from Control?
- What conditions should cause the vehicle to stop?
- How will the emergency-stop signal override normal operation?
- What happens if sensor information is unavailable?

Example Safety Rule

If:

obstacle_distance < safe_distance

Then:

emergency_stop = TRUE

You do **not** need to fully assemble or wire the vehicle for Milestone 1.

You are developing the plan for how these pieces will eventually connect.

What Each Team Must Submit

Each team should submit **one document** containing the following five items.

1. Simple Subsystem Diagram

Create a block diagram showing:

Inputs → Your Subsystem → Outputs

Keep it simple.

You do not need a detailed electrical schematic.

---

2. Input/Output Table

List the important information entering and leaving your subsystem.

At minimum include:

Signal Source Destination Description Data Type Units

Add expected range, update rate, and fault behavior when known.

You do not need to document every internal MATLAB variable.

Focus on information that is exchanged **between teams**.

---

3. Team Responsibilities

List each team member and identify their primary responsibility.

For example:

Student	Initial Responsibility
Student A	Sensor/interface research
Student B	Simulink model planning
Student C	Testing and interface documentation

Responsibilities can change later, but everyone should have an initial technical role.

---

4. Team Dependencies

Identify which teams you need to communicate with.

For example:

Motion Planning receives obstacle information from Perception and sends desired speed and steering information to Vehicle Control.

---

5. Initial Testing Plan

In a short paragraph, explain how you might test your subsystem later.

You do not need to perform the test now.

Example:

The Perception team will provide simulated obstacle-distance measurements to the subsystem. We will verify that valid obstacle information is produced and sent to the Safety subsystem. We will also test what happens when sensor information is missing or invalid.

Example of One Complete Interface

Here is an example of the level of detail expected.

Signal Name: obstacle_distance

Source: Perception

Destination: Active Safety

Description: Estimated distance between the vehicle and the closest obstacle

Data Type: double

Units: meters

Expected Range: approximately 0–4 meters

Update Frequency: approximately 10 Hz

Invalid/Fault Condition: sensor_valid = FALSE

If Information Is Unavailable: Vehicle should enter a safe-stop condition

That is enough for Milestone 1.

You are **not expected to implement this signal yet**.

Class-Level Deliverable

After each team identifies its interfaces, the class will combine them into one overall system diagram.

Our starting architecture is:

Sensors → Perception → Motion Planning → Vehicle Control → Motors/Steering

with:

Active Safety → ability to override Control/Hardware

The purpose of this diagram is to make sure that the four teams are building parts of **one system**, rather than four unrelated projects.

Definition of Done

Milestone 1 is complete when:

- Your subsystem has a clear purpose.
- You have identified its important inputs.
- You have identified its important outputs.
- Each input has a known source.
- Each output has a known destination.
- Important units and data types have been identified.
- You have discussed shared signals with the teams that send or receive them.
- You have created a simple subsystem block diagram.
- Each student has an initial responsibility.
- You have included a basic plan for how the subsystem can eventually be tested.
- Any remaining unknowns are clearly identified.

You are **not required to have every technical detail of the final vehicle solved at this stage**.

After Milestone 1

The interface document becomes our **starting design** for Milestone 2.

Interfaces can still be changed if testing shows that a change is necessary. However, any change that affects another subsystem must be discussed with the affected team so that both sides remain compatible.

Most Important Thing to Remember

Milestone 1 is a system-design milestone, not an implementation milestone.

You are not being asked to solve autonomous driving by Friday.

You are being asked to answer:

What does my subsystem receive?

What does my subsystem produce?

Who do I need to communicate with?

If you can clearly answer those three questions, you are on the right track.